

Jev Decision Layer — Design Spec

Date: 2026-09-27 **Branch base (planned):** working tree @ [b96c113](#) (current tip of [trading-sentinel](#)) **Working branch (planned):** [feat/jev-decision-layer](#) **Author:** Hermes + Uru collaborative (brainstorm 2026-09-27) **Status:** Draft — awaiting Uru approval

1. Executive summary (one paragraph)

Trading Sentinel today pays MiniMax (LLM) for ~3 narrow, deterministic-feeling jobs: 8-category news classification, the advisory verdict vocabulary, and the optional-ai async review layer. Each call takes 1-15 seconds and costs 100-200× more than necessary. Jev (TypeSafe AI's System One model) is a *fundamentally different* primitive: not an LLM, generates no text, returns typed decisions (Choice / Score / Noul) with calibrated probabilities in ~150ms. **We are adding a Jev decision layer in M4 read-only mode first** — it annotates every closed trade with 3 atomic quality Scores, every signal evaluation with a Noul plausibility check, and replaces the news_classifier 8-category decision — all without touching the order path. After 2-4 weeks of paper data, the highest-confidence M4 use case is promoted to M2/M3 (operator-confirmation soft veto). The M4 layer itself is a measurement instrument; you earn the right to make decisions from it later.

Cost impact: news classification 100× cheaper, per-trade grading \$0.05/day, signal annotation \$0.02/day. **Total: <\$1/day** to run the entire M4 layer.

Risk: zero order-path impact. Fail-closed throughout. The LLM stays as fallback when Jev confidence is low.

2. Motivation and goals

2.1 Why now

Three converging observations from the 2026-09 codebase deep-read:

1. **The system already has 12+ small-judgment call sites that today are LLM-based or heuristic.** Examples: [agent/news_classifier.py](#) (8-category LLM call with 1s timeout), [agent/advisory.py](#) (verdict vocabulary: APPROVE / APPROVE_WITH_CONCERNS / REVIEW_UNAVAILABLE / REJECT), [fno_gates.py](#) (12 hard if/else gates, no notion of "how close was the call"). Each is a small judgment, not open-ended reasoning.

1. **Jev is the right shape.** Per TypeSafe's launch blog, Jev is built specifically for this class of problem: "*A System One model makes fast, structured decisions that software can use directly.*" Their reference trading implementation ([jarrodwatts/jev-trader](#), 2,400+ stars) makes one buy/sell decision per 300ms block on a crypto order book — exactly the same pattern as our penny scanner ticks every 30 seconds.

1. **The cost is dramatically lower.** \$0.042 per million input tokens, output free. ~150ms latency. A combined MiniMax + Jev stack uses the right model for the right job: MiniMax for prose/strategy/multi-step reasoning, Jev for the routine classification/scoring/guardrail decisions.

2.2 Goals

1. **M4 first — read-only annotation, no order-path impact.** Surface per-trade quality scores, signal plausibility, and a faster news classifier. Earn the right to make decisions from these.
2. **Replace the 8-category news_classifier decision with Jev** (cheapest immediate win, surfaces the integration pattern for everything else).
3. **One Jev client, multiple call sites.** A `jev_client.py` module owns auth, fail-closed, retries, and calibration; the trade grader, signal annotator, and news classifier all call into it.
4. **Phase 2 M2/M3 promotion with operator sign-off.** After ≥ 4 weeks of paper data, the highest-confidence use case becomes a soft veto requiring operator confirmation.
5. **Contract health for Jev specifically.** A new `agent/contract_health` invariant: Jev responses are schema-valid, calibrated probabilities, and the pinned model version hasn't drifted.

2.3 Non-goals (explicit, for the pitch)

- **NOT replacing MiniMax.** MiniMax stays for strategy reasoning, agent verdicts with prose rationale, and any open-ended generation. Jev augments, doesn't replace.
- **NOT changing the order path in Phase 1.** Every M4 call is *annotation only*. Operators see new columns in the hourly report; no signal is rejected, no position is closed, no order is cancelled by Jev.
- **NOT replacing `fno_gates.py`'s hard rules.** The hard caps (max_loss, sizing, kill switches) stay. Phase 2 may add a *confidence overlay* but the hard rules are not touched.
- **NOT refactoring the existing LLM call sites wholesale.** Only the 8-category news_classifier decision is swapped. The verdict vocabulary, async review, and contract health stay on LLM.
- **NOT enabling any new live trading.** Phase 1 is M4 read-only; even Phase 2's M3 promotion is operator-confirmation, not autonomous.



3. What Jev actually is (one paragraph, for the pitch)

Jev is TypeSafe AI's "System One" model — launched 2026-09-15, \$40M seed led by DCVC, founded by Diego Almeida (ex-OpenAI co-founder). It is **not an LLM** and generates no text. You send it a `state` (any text or JSON blob) plus a set of typed `questions`, and it returns typed answers drawn from the schema you defined, with calibrated probabilities and a confidence score. Three primitives: `Choice` (pick from ≤ 255 options, returns probability per option + confidence), `Score` (rate against 2-10 ordered levels), `Noul` (yes-no probability). It cannot hallucinate the *format* of an answer (the schema constrains it) but it can be wrong on *content* — same caveat as any classifier. ~150ms latency, \$0.042/MTok input, output free. The "magic" is **calibration**: when Jev says 0.9, the empirical accuracy is ~90% — the probability is meaningful as a business signal, not a vague confidence.

Concrete example: ask Jev "Will MON-USDC be higher or lower in 100 blocks?" with state = current order book → it returns `{"buy": 0.77, "sell": 0.23}` in 150ms. Vercel swapped a Gemini 2.5 Flash classifier for Jev and got 6× faster, saturating their eval.



4. The plan — what changes, where, in what order

4.1 New modules

File	LOC est.	Role
<code>python-engine/jev_client.py</code>	~250	Thin wrapper around <code>POST https://api.typesafe.ai/v1/systemone</code> . Auth via <code>TYPESAFE_API_KEY</code> env var. Schema-validated response. Fail-closed on missing key, network error, or schema mismatch. Batched multi-question calls. Optional pydantic models for Choice/Score/Noul answers.
<code>python-engine/trade_grader.py</code>	~350	Called from <code>position_tracker.mark_position_closed</code> (or equivalent) for every penny / momentum / FNO close. Sends 3 atomic <code>Score</code> questions over the closed trade state. Persists to new <code>trade_grades</code> table.
<code>python-engine/signal_annotator.py</code>	~300	Called from <code>penny_scanner._evaluate_ticker_*</code> and <code>fno_orchestrator._try_entry_for_leg</code> for every signal evaluation (accept OR reject). Sends 1 <code>Noul</code> (plausibility) + 1 <code>Choice</code> (signal class). Persists to new <code>signal_annotations</code> table.
<code>agent/news_classifier.py</code> (modify)	+30	Replace the 8-category LLM call with a Jev <code>Choice</code> call. Keep the LLM call as fallback when Jev confidence < 0.7 OR when the LLM is needed for rationale prose.
<code>agent/contract_health.py</code> (modify)	+20	Add 1 new invariant: Jev schema + calibration + pinned-model check.

4.2 New SQLite tables

```
CREATE TABLE trade_grades (  
  position_id INTEGER NOT NULL,
```

```

source TEXT NOT NULL, -- PENNY_PAPER / MOMENTUM_PAPER / FNO_PAPER etc.
ticker TEXT NOT NULL,
leg TEXT NOT NULL, -- MIS / CNC / CE / PE
entry_timing_score REAL, -- 0-2 from Jev Score
exit_reason_score REAL, -- 0-2 from Jev Score
structure_quality_score REAL, -- 0-2 from Jev Score
entry_timing_confidence REAL,
exit_reason_confidence REAL,
structure_quality_confidence REAL,
model_version TEXT NOT NULL, -- e.g. jev-1.13
fingerprint TEXT NOT NULL, -- SHA-256 of state dict
raw_answers_json TEXT NOT NULL, -- full Jev response for replay
created_at TEXT NOT NULL,
PRIMARY KEY (position_id)
);

```

```

CREATE TABLE signal_annotations (
scan_id TEXT NOT NULL,
ticker TEXT NOT NULL,
leg TEXT NOT NULL,
accepted INTEGER NOT NULL, -- 0/1, mirrors the signal's accept flag
plausibility_noul REAL, -- 0-1, Jev's Noul
plausibility_confidence REAL,
signal_class TEXT, -- e.g. "noise", "real_breakout", "duplicate"
signal_class_probabilities_json TEXT, -- full probability dict
model_version TEXT NOT NULL,
fingerprint TEXT NOT NULL, -- SHA-256 of signal state
raw_answers_json TEXT NOT NULL,
created_at TEXT NOT NULL,
PRIMARY KEY (scan_id, ticker, leg)
);

```

4.3 New operator-facing surfaces

- **Hourly digest** (existing `notify_operator`): add 3 new lines:
- `top_plausible_signals_today`: N (avg plausibility=X.XX)
- `lowest_graded_trades_today`: N (avg structure_quality=Y.YY)
- `news_classifier_jev_calls_today`: N (avg confidence=Z.ZZ)
- **Dashboard** (node-gateway client): add 2 new panels:
- "Trade quality distribution" — histogram of `structure_quality_score` over the last 30 days
- "Signal plausibility vs realized" — scatter of `plausibility_noul` vs final R-multiple

4.4 Implementation order (the work plan)

Week 1 (scaffold + cheapest integration)

1. `python-engine/jev_client.py` — full wrapper, fail-closed, schema-validated, pydantic models. Unit tests for happy path + every failure mode.
2. `agent/contract_health.py` — Jev invariant. Test runs the invariant against a real call (or a recorded fixture).
3. `agent/news_classifier.py` — swap the 8-category decision to Jev. Keep MiniMax as fallback. Unit tests for both code paths.
4. `config.py` — add `JEV_API_KEY` (optional), `JEV_MODEL` (default `jev-latest`), `JEV_TIMEOUT_MS` (default 500), `JEV_MIN_CONFIDENCE` (default 0.6).

Week 2 (read-only annotation)

1. `trade_grader.py` — module + tests + integration call from `position_tracker.mark_position_closed` (and FNO equivalent).
2. `signal_annotator.py` — module + tests + integration call from `penny_scanner` and `fno_orchestrator`. The integration is a *try/except* wrap that logs the failure and never affects the signal decision.
3. `db/schema.sql` — add `trade_grades` and `signal_annotations` tables with idempotent DDL.
4. Hourly digest surfaces — add the 3 new lines to the existing `notify_operator` payload.

Week 3 (observability + dashboard)

1. `python-engine/ops_liveness_daily.py` (or equivalent) — extend to include the 3 new metrics.
2. Node-gateway client — 2 new dashboard panels. (Frontend work; can be deferred to week 4 if needed.)
3. End-to-end paper run for 7 days, gather evidence.

Week 4-6 (paper-data observation)

1. NO code changes to order path. Operators review the daily distributions.
2. After ≥ 4 weeks of paper data, write a *promotion memo* — "which of the 3 M4 use cases should become M2/M3?" — based on observed evidence, not theoretical.

Phase 2 (M2/M3 promotion, only after paper evidence)

1. Most likely candidate: `signal_annotator.duplication_check` (M3) — operator confirmation required when Jev flags a redundant signal with confidence > 0.85 .
2. Other candidates (M1 only): `trade_grader.exit_reason` post-mortem in the morning brief.
3. Last: `fno_gates` confidence-aware variant (M2) — most invasive, requires spec revision + re-derivation of every gate.

4.5 The advantages over the current system (for the pitch)

What today happens	What Jev does instead	Why it matters
MiniMax classifies news into 8 categories in 1-15 seconds with <code>fail-closed</code> returning UNKNOWN	Jev <code>Choice</code> over 8 categories in ~ 150 ms with calibrated probability per category	100× cheaper, 10× faster, probabilities are empirical (when Jev says 0.9, it's right 90% of the time)
No per-trade grading beyond realized P&L;	3 atomic <code>Scores</code> per closed trade: entry timing, exit reason, structure quality	Surfaces "entries are getting worse" <i>before</i> the next drawdown — a pattern that's invisible today
No signal plausibility check beyond deterministic gates	<code>Noul</code> over the signal's full state dict	Catches the <i>redundant-signal storm</i> anti-pattern before it inflates <code>ops_funnel_daily</code>

What today happens	What Jev does instead	Why it matters
Heuristic accept/reject, no confidence margin	Jev returns a Choice over the strategy's top-3 picks	Future: "Jev was 60% confident this was a real signal" is auditable in the same row
Manual post-mortem on edge cases	Structured per-trade annotation that ages into a training set	The same data can later train a custom Jev-style model on our specific strategies

The structural point: today, every classification is either a hardcoded rule (fast, rigid, can't say "I'm not sure") or an LLM (slow, expensive, can't say what it doesn't know). Jev sits in the middle — fast and cheap like a rule, calibrated and typed like a model. That's the slot our 12 small-judgment call sites have been waiting for.



5. Risk and rollback

5.1 What can go wrong in Phase 1

Risk	Probability	Mitigation
Jev API outage during a paper run	Medium	fail-closed returns JEV_DISABLED ; annotation skipped; signal/order path unaffected. Hourly digest shows "Jev: 0 calls today (DISABLED)".
Jev schema returns something our pydantic doesn't accept	Low	Strict pydantic validation. On mismatch, log full response at WARNING and skip the annotation.
Jev calibration drift (e.g. says 0.9 but is right 60%)	Low	agent/contract_health runs a weekly calibration check on a held-out paper signal set. Surfaces drift in the hourly digest.
Pinned model version changed by TypeSafe	Low	JEV_MODEL_PINNED_VERSION env var; contract_health asserts the active model matches. Operator gets paged on mismatch.
Latency on Jev call spikes during a scan tick	Low	500ms timeout; all annotation calls are fire-and-forget (off the order path).

Risk	Probability	Mitigation
Cost overrun from accidentally calling on a hot loop	Very low	Per-call counter in ops_liveness_daily ; budgeted at <\$1/day. Contract_health pages on >\$5/day.

5.2 Rollback plan

- All Phase 1 modules are additive. To roll back: set [JEV_ENABLED=False](#) (or omit [JEV_API_KEY](#)). All annotation stops; the order path is unchanged.
- The news_classifier swap is the only Phase 1 change to existing code; rolling back is restoring the prior LLM call (1-line revert).

5.3 What does NOT need a roll-back plan

- Nothing in the order path changes. The risk of a Phase 1 regression breaking trading is the same as today's risk: zero, because nothing in the order path changes.



6. Testing strategy

6.1 Unit tests (existing pytest harness)

- [test_jev_client.py](#): every failure mode (no key, network error, 4xx, 5xx, schema mismatch, timeout). One happy-path test against a recorded fixture.
- [test_trade_grader.py](#): state construction + table write. Mocked Jev responses for each Score level.
- [test_signal_annotator.py](#): same shape, plus the "annotation failed but signal still evaluated" path.
- [test_news_classifier_jev.py](#): 8 categories happy path, low-confidence fallback to LLM, schema mismatch.

6.2 Integration tests (against recorded fixtures)

- One recorded Jev response per question type (Choice / Score / Noul). Tests use the fixture, not the live API.
- Calibration check test: a hand-built 100-row dataset, verify the contract_health check correctly flags drift > 5%.

6.3 Paper run (manual)

- 7-day paper run with [JEV_ENABLED=True](#). Operators check the hourly digest daily. Verify:
- No order-path impact (entry count matches non-Jev baseline)
- Jev availability > 95% of the day
- News classifier latency p50 < 200ms, p95 < 500ms
- Trade grades arrive within 5s of position close

- No spurious JEV_DISABLED warnings

6.4 What we do NOT test

- Live trading impact (none — Phase 1 doesn't touch live).
- Cross-asset-class behaviour (each subsystem is tested independently).
- TypeSafe's API itself (we test our wrapper against it, not TypeSafe's correctness).



7. Open questions and decisions taken under user delegation

7.1 User delegated

- "I can leave that decision to you" — which of the 3 M4 integrations to build first. **My call: build all three in the same work pass.** They share the `jev_client.py` primitive; doing one without the others is wasteful. Order: news_classifier swap (cheapest) → trade_grader → signal_annotator.
- "I allow you to take a call on it, we have 2 ai and how to use it" — MiniMax vs Jev split. **My call: MiniMax for prose/strategy/multi-step (unchanged), Jev for routine classification/scoring/guardrails (new).** Concretely: only the 8-category news_classifier decision is swapped; verdict vocabulary, async review, and contract_health stay on LLM.

7.2 Open and unanswered

- **Jev model pinning** (timed out on user). **My call: env var override, default `jev-latest`, `contract_health` test asserts the active model matches a pinned date-stamped version the operator sets.** This gives operator control without code changes; the operator can roll forward at their own pace.
- **Phase 2 M2/M3 promotion order** (deferred until ≥ 4 weeks of paper evidence). My provisional ranking, subject to data: `signal_annotator.duplication_check` (M3) > `trade_grader.exit_reason` (M1) > `fno_gates` confidence overlay (M2).

7.3 Awaiting sign-off

- The spec itself.
- The 3-M4-in-one-pass scope.
- The 4-week paper observation gate before Phase 2.
- The "operator pins model version" decision.



8. Success criteria

8.1 Phase 1 (M4 read-only) — ship gate

- All 3 M4 modules live, fail-closed, tested.
- News classifier: at least 7 days of paper data with Jev calls, MiniMax fallback < 20% of calls.
- Trade grader: > 95% of closed trades have grades within 5s of close.
- Signal annotator: > 95% of evaluations have annotations within 5s of scan tick.

- No order-path regressions (entry count, P&L, kill switches — all match pre-Phase-1 baseline).
- Daily Jev cost < \$1.
- Hourly digest surfaces the 3 new metrics.

8.2 Phase 2 (M2/M3 promotion) — gate before any code change

- ≥4 weeks of paper data.
- Calibration check on trade_grader data: Brier score < 0.20 on entry_timing_score, exit_reason_score, structure_quality_score.
- Operator review of the promotion memo.
- Spec revision for the specific M2/M3 use case (this spec is for Phase 1 only).



9. Appendix: Jev model-pinning recommendation detail

9.1 The trade-off

- **Pin to a date-stamped version (e.g. `jev-1.13`):** reproducible. Audit trail. But the model becomes stale; if TypeSafe ships a clearly-better 1.14 with a security fix, you're stuck.
- **Pin to `jev-latest`:** always current. But a silent model update can change the calibration of your historical grades, and your dashboards will show "drift" that isn't real.
- **Operator-pinned date-stamped version with explicit roll-forward:** best of both. Operator sets `JEV_MODEL_PINNED_VERSION=jev-1.13` in env. `Contract_health` checks the active model matches. Operator rolls forward at their own pace by editing env and restarting.

9.2 My recommendation (call made under user delegation)

- `JEV_MODEL = "jev-latest"` (the default for new deployments).
- `JEV_MODEL_PINNED_VERSION = "jev-1.13"` (the operator-pinned "ground truth").
- `agent/contract_health` invariant: `active_model == JEV_MODEL_PINNED_VERSION OR (active_model != pinned AND operator_acknowledged_recently)`.
- If they don't match, the Jev client logs a WARNING on every call AND `contract_health` pages once per day.
- Operator "acks" the new version by updating `JEV_MODEL_PINNED_VERSION` to the new alias.

This pattern: **fail-soft on the running model, fail-loud on the contract** — same philosophy as every other config in the codebase.



End of spec. Awaiting Uru approval before transitioning to writing-plans.